Liber Usualis - Architecture

1. Overview

Liber Usualis is a data-driven liturgical text generation system for the Roman Catholic Divine Office. Given a date and an occasion, it selects the governing liturgical tag sets, composes tagged content, and can superimpose translations and chant. The currently supplied Roman Breviary data follows the 1888 edition.

Tags are the central domain model. Liturgical choices should normally be represented by tagged content, category files, discrimination tables, annual and daily rule tables, or implication rules rather than feast-specific branches in Python or JavaScript.

1.1. Technology Stack

Layer	Technology
Backend	Python 3.13, Bottle, Waitress
Frontend	Alpine.js 3.x, jQuery, Exsurge.js
Templates	Bottle templates
Data	Tagged JSON plus untagged text and GABC resources
Infrastructure	Nix flakes and native nixpkgs image builders

2. Backend

2.1. Module Boundaries

The request-to-content flow is divided across the following modules:

- `server.py` parses HTTP parameters, invokes domain operations, and serializes responses.
- Root `datamanage.py` constructs the configured corpora and orchestrates rite requests, office selection, translation, and chant.
- `kalendar/kalendar.py` constructs and prioritizes the annual kalendar.
- `kalendar/daily_tagger.py` applies diurnal, Vesperal, Martyrology, votive, and implication rules for a particular date.
- `kalendar/datamanage.py` loads kalendar data and caches annual results.
- `composer/book.py` loads one book's tagged, category, discrimination, and untagged resources.
- `composer/corpus.py` searches across books and recursively composes rites.
- `composer/rite.py` superimposes contingent content onto a composed rite.
- `composer/psalms.py` resolves psalm references in untagged files.
- Root `breviarium.py` is a CLI and test-facing consumer of this pipeline. It is not the composition engine.

The `Bookshelf` boundary in `bookshelf/bookshelf.py` exposes `book_srcs()` to kalendar code. Both `Corpus` and the kalendar's lightweight corpus implementation satisfy that interface, so `kalendar/` does not depend on `composer/`.

2.2. HTTP API: `server.py`

2.2.1. `GET /day`

Parameters:

- `date` - ISO date, `YYYY-MM-DD`
- `time` - `vesperale` or `diurnale`
- `votives` - plus-separated enabled votive tags; an empty value is allowed

The route calls `daily_tagger.get_vespers()` or `daily_tagger.get_diurnal()`. It returns:

- `tags` - all resolved tag sets
- `primary` - the primary name and tag set
- `commemorations` - ranked name/tag-set pairs
- `omissions` - ranked omitted name/tag-set pairs
- `commemoratio-matutini` - the Matins commemoration, when present

2.2.2. `GET /title`

Parameters are `date`, `hour`, `select`, and `votives`. The route applies the same office-selection adjustment used by rite generation and returns the selected primary name and tag set. The frontend uses this before rendering a group of rites.

2.2.3. `GET /rite`

The preferred frontend form sends:

- `date`
- `rites` - underscore-separated requests; each request is `what.noending.select`
- `translation` - `none`, `english`, or `deutsch`
- `privata` - `privata` for private recitation
- `chant` - `true` to apply chant
- `votives` - plus-separated votive tags
- `version` - the expected ritegen/CSS resource version

The packed form returns a list containing one result per requested rite. A legacy single-rite form using `hour`, `select`, and `noending` is also accepted and returns one result object directly. Each result contains `rite`, `used-primary`, `used-commemorations`, and `commemoratio-matutini`.

2.2.4. Other Endpoints

- `GET /kalendar` returns the navigation skeleton and display kalendar.
- `GET /chant/<path>` serves generated chant resources.
- `GET /resources/<path>` serves frontend resources.

2.3. Stage 0: Books and Corpora

Repository-managed books live directly under `data/`. Externally maintained books are synchronized into `data/generated/` by `sync-books.py` according to the optional repository-root `books.json`. Both paths are ignored by Git, so synchronization is a separate operator step rather than part of a pure build.

A `Book` identifies a source directory and a book-title tag. A `Corpus` searches one or more primary books. A `ContingentCorpus` retains the primary-book context while searching separate content books, which is how translations and chant are applied without changing the structure of the Latin rite.

Root `datamanage.py` currently constructs:

- `DEFAULT_CORPUS` from the 1888 breviary and 1846 Martyrology
- `ENGLISH_CORPUS` and `DEUTSCH_CORPUS` from translation books
- `CHANT_CORPUS` from generated chant books

2.4. Stages 1 and 2: Kalendar and Daily Tagging

2.4.1. Annual Kalendar

`kalendar/kalendar.py` assembles each book's kalendar sources into their normal dates, applies the annual `tabella.json` rules, resolves precedence and translations of feasts, and caches the completed year through `kalendar/datamanage.py`.

Annual rule processing and daily rule processing are separate responsibilities. The annual engine must remain usable through the `Bookshelf` interface rather than importing composer classes.

2.4.2. Daily Tagging

Daily rule tables live in `data/{book}/kalendarium/`:

- `tabella-diurnalis.json` controls the daytime office.
- `tabella-vesperalis.json` resolves first and second Vespers.
- `tabella-martyrologii.json` controls the Martyrology reading.
- `sequentes.json` adds implied tags.

Rules use `include` and `exclude` restrictions and declarative operations such as `crea`, `combina`, `mutate`, `duplicate`, `dele`, and `errora`.

`get_vespers(book, day, votives)` resolves the overlap between the current day's second Vespers and the next day's first Vespers. `get_diurnal(book, day, votives)` applies daytime and Martyrology rules, including the next day's lunar tag for Martyrology. Their output is a sequence of service tag sets describing the primary office, commemorations, omissions, and applicable votives.

2.5. Stage 3: Composition

2.5.1. Loading Tagged Content

`Book.get_tagged_file()` loads JSON entries from a book's `tagged/` tree and adds the book title to every tag set. The normal entry shape is:

```
{
  "tags": ["immaculata-conceptio-bmv", "nomen"],
  "datum": "Immaculatae Conceptionis B.M.V."
}
```

`tags` may also be a list of tag lists when the same datum belongs to multiple contexts. Any key other than `tags` and `datum` is expanded into a separate entry whose tags include that key. This supports compact structured source data without treating extra keys as arbitrary metadata.

Chant is not fetched from `src` values in primary data. It is tagged content in a contingent corpus and is attached during superimposition.

2.5.2. Recursive Composition

`Corpus.compose(item, selected, alternates)` is the public composition entry point and returns a `Rite`. It delegates recursion to `Corpus._process()`:

1. A set of tags is a search request.
2. Contradicted positional tags are removed from the selected context.
3. An explicit `occurrens` reference or a matching alternate can switch the active feast context.
4. Requested temporal tags replace contradicted temporal context.
5. `Corpus.search()` resolves the resulting query.
6. Lists under `datum` are recursively composed; strings are returned as text, with saint-name substitution where required.

A query containing `commemorationes` is handled specially: one `formula-commemorationis` is composed per ranked commemoration and a shared terminating collect is appended.

2.5.3. Search and Discrimination

`Corpus.search(query)` resolves content as follows:

1. A tag beginning with `/` is treated as an untagged-resource reference. The primary `Corpus` resolves these through `composer/psalms.py`; `ContingentCorpus` recognizes `/psalmi` references and otherwise retrieves a matching untagged resource from its content books.
2. The corpus builds a pile from files named by the query plus the default pile names, across all primary books.
3. Book-title tags are added to the query.
4. Entries whose tags are subsets of the query become candidates.
5. Rules from `discrimina/general.json` are applied in order. Whenever at least one candidate matches a rule, nonmatching candidates are discarded.
6. A sole candidate is returned immediately. Otherwise candidates are ordered by tag count; a uniquely more specific candidate wins, and a remaining tie raises an ambiguity error unless the caller explicitly requested multiple results.

`Corpus.discriminate(table, tags)` is a separate bitmask ranking operation used where complete tag sets, such as commemorations, must be sorted. It is not the candidate-filtering implementation used by `Corpus.search()`.

2.5.4. Untagged Resources and Psalms

Untagged resources live under each book's `untagged/` tree and may be JSON, text, or GABC. `Book.retrieve_untagged_file()` verifies that resolved paths stay inside that tree. Psalm references such as `/psalmi/psalmus-i` are interpreted by `composer/psalms.py`, which extracts the requested portion of the untagged psalm text.

2.6. Stage 4: Superimposition

`Rite.superimpose(corpus, tag)` deep-copies the composed rite and traverses its tagged nodes. It searches the supplied `ContingentCorpus` using each node's `quaesitum` or tags and attaches matching content under `tag`. Root `datamanage.rite_request()` uses this operation to attach translations under `translation` and chant under `cantus`, returning a new `Rite` each time.

Translation books remain parallel to source content through matching query tags:

- `data/breviarium-1888-english/`
- `data/breviarium-1888-deutsch/`

Generated chant books live under `data/generated/`. The frontend receives translated and chanted content inline in the rite tree; it does not perform a second translation lookup.

3. Frontend

3.1. Page Shell and State: `web/templates/pray.tpl`

The Bottle template loads localized page text and the versioned pray resources. Its Alpine `x-data` object owns the interactive state, including:

Property	Purpose
`calendarDate`	Civil date currently being viewed
`liturgicalDay`	Resolved `/day` response
`hour`	Selected occasion identifier
`parameters`	Persisted office, recitation, and translation preferences
`rite`	Rendered rite markup
`nextOccasion`	Persisted next occasion and date

The template methods coordinate date changes, occasion changes, incrementing to the next occasion, and asynchronous refreshes. Alpine watchers refresh the day or rite when date and preference state changes.

3.2. Ritual Structure and Requests: `web/resources/pray/js/ambit.js`

`Rite`, `Occasion`, and `Ambit` describe which backend rites make up a displayed occasion. `fullAmbit` defines the normal sequence from Matins through Compline; `SingleAmbit` and dedicated ambits represent votive offices and other rites.

`defineAmbit()` supports the daily office, Little Office, Office of the Dead, Gradual and Penitential Psalms, recommendation of a soul, the indulgence at the point of death, grace at meals, the clerical itinerary, and modes that include or isolate the Little Office.

`Ambit.riteList()` filters an occasion against the resolved day tags. A small number of existing frontend suppression rules also prevent incompatible appended rites, such as the seasonal BVM antiphon after specified offices. New liturgical behavior should prefer backend rule data where the tag pipeline can represent it.

`getLiturgicalDay()` calls `/day`. `getRite()` calls `/title`, converts the selected occasion into packed rite requests, calls `/rite` once, and passes each result to the renderer.

3.3. Rendering and Window Helpers

`web/resources/pray/js/ritegen.js` converts structured rite responses into HTML and recognizes inline `translation` and `cantus` content.

`web/resources/pray/js/psalmify.js` formats psalm text. Files under `web/resources/pray/js/chant/` render GABC lazily with Exsurge and provide Latin hyphenation support.

`web/resources/pray/js/pray-window.js` maps UI locales to translation books and handles responsive panel sizing. It does not own the primary Alpine event flow.

4. Tagging System

4.1. Categories

Tags are lowercase, hyphen-separated strings. Category definitions live in `data/{book}/categoriae/`:

- `positionales.json` defines mutually exclusive positions and contexts.

- `temporale.json` defines seasons, ranks, propers, and commons.
- `proprium-sanctorum.json` identifies feasts of saints.
- `objecta.json` identifies content types.

Category entries beginning with `/` reference another category and are expanded recursively by `Corpus.expand_cat()`. Queries must not silently retain mutually exclusive tags from category groups.

4.2. Runtime Tag Flow

1. Annual kalendar lookup produces the base occasions for the date.
2. Daily tables resolve the primary office, commemorations, omissions, and votives.
3. `sequentes.json` adds implied tags.
4. `datamanage.adjust_tags()` currently applies explicit selection of the Little Office, Office of the Dead, or seasonal BVM antiphon. This is an acknowledged hard-coded boundary and should not be expanded when data rules can express the behavior.
5. Composer combines the selected context with item tags and searches by subset.

4.3. Data Layout

```
data/
|-- breviarium-1888/
|   |-- categoriae/
|   |-- discrimina/
|   |-- kalendarium/
|   |-- tagged/
|   `-- untagged/
|-- breviarium-1888-english/
|-- breviarium-1888-deutsch/
|-- martyrologium-1846/
`-- generated/
```

5. End-to-End Example

A normal frontend request for Matins follows this sequence:

```
1. pray.tpl initializes Alpine state for the selected civil date.
2. ambit.js requests /day?date=...&time=diurnale&votives=....
3. daily_tagger returns resolved service tag sets.
4. ambit.js filters the Matins Occasion and requests /title.
5. ambit.js sends one packed /rite request for the included Rite items.
6. datamanage.rite_request() selects the primary and alternate tag sets.
7. Corpus.compose() recursively searches tagged content and returns a Rite.
8. Requested translation and chant corpora are superimposed on that Rite.
9. server.py serializes the result; ritegen.js renders each returned rite.
```

6. Infrastructure

`flake.nix`, `flake.lock`, `pyproject.toml`, and `uv.lock` remain at the repository root because they describe the complete source and Python workspace. Root `.envrc` and `.python-version` provide project-level tool discovery.

Deployment modules and provider policy files live under `nix/`. The flake passes its tracked source to `nix/nixos-config.nix` for installation under `/var/lib/libu`. Deployment preserves an operator-provided `books.json` and `data/generated/` while replacing the tracked application. Operators must run `sync-books.py` separately when generated books need to be installed or updated. `nix/image-formats.nix` customizes nixpkgs' native image variants. The `libu-images` NixOS configuration exposes Linode, Proxmox, Amazon, application ISO, and installer ISO images through `system.build.images` and `nixos-rebuild build-image`. Package aliases preserve the corresponding `nix build .#libu-*` commands. Docker output is not provided. The separate Linode hardware module backs the installed host's `nixos-rebuild` configuration and is not included in generated images because its filesystem UUIDs are host-specific.

7. Key File Index

File or directory	Purpose
`server.py`	HTTP routing and serialization
`datamanage.py`	Corpus setup and request orchestration
`breviarium.py`	CLI and test-facing composition consumer
`bookshelf/bookshelf.py`	Kalendar-facing book-source boundary
`kalendar/kalendar.py`	Annual kalendar and annual rules
`kalendar/daily_tagger.py`	Daily, Vesperal, and Martyrology rules
`kalendar/datamanage.py`	Kalendar loading and annual cache
`composer/book.py`	Book data loading and entry expansion
`composer/corpus.py`	Search, discrimination, and composition
`composer/rite.py`	Translation and chant superimposition
`composer/psalms.py`	Psalm reference resolution
`web/templates/pray.tpl`	Page shell and Alpine state
`web/resources/pray/js/ambit.js`	Ritual structure and HTTP request composition
`web/resources/pray/js/ritegen.js`	Rite rendering
`web/resources/pray/js/pray-window.js`	Window and panel helpers
`data/{book}/tagged/`	Tagged liturgical content
`data/{book}/untagged/`	Untagged text and GABC resources
`data/{book}/categoriae/`	Tag categories and contradiction groups
`data/{book}/discrimina/`	Search and rank rule tables
`data/{book}/kalendarium/`	Annual, daily, and implication rules
`flake.nix`	Development and deployment outputs
`nix/nixos-config.nix`	NixOS service and nginx configuration
`nix/image-formats.nix`	Native image-variant customization